

Machine Learning Project Report

Louca Malerba, Antoine Mazet, Mathys Piet

Centrale Casablanca

Contents

1	Introduction	1	5.2	Expériences avec U-Net	14
1.1	Contexte	1	5.2.1	Exploration d'architectures alternatives	14
1.2	Objectifs	2	5.2.2	Optimisation de la fonction de perte	14
2	Présentation du dataset	2	5.2.3	Configuration finale retenue	15
3	Présentation de U-Net	4	5.3	Variantes de U-Net	15
3.1	Architecture U-Net	4	5.3.1	Attention U-Net	15
3.1.1	Chemin contractant (encoder)	4	5.3.2	Nested U-Net avec Squeeze-and-Excitation	15
3.1.2	Chemin expansif (decoder)	4	5.3.3	Entraînement complet	16
3.1.3	Convolutions	5	5.4	Pipeline de spécialisation par clustering	16
3.1.4	Max Pooling	5	5.4.1	Clustering K-means	16
3.1.5	Upsampling	6	5.4.2	Classification CNN pour prédiction de cluster	17
3.1.6	Upsampling	6	5.4.3	Modèles U-Net spécialisés par cluster	17
3.2	Variantes	6	5.4.4	Aggrégation des résultats	18
3.2.1	U-Net++ (Nested U-Net)	6	6	Conclusions	18
3.2.2	Attention U-Net	7	1	Introduction	
4	Méthodologie	8	1.1	Contexte	
4.1	Vue d'ensemble	8	La segmentation est l'une des applications du machine learning les plus importantes dans le domaine médicale. La segmentation sémantique, c'est-à-dire la catégorisation de pixels individuels, représente 70% des challenges du bio-médical autour de la segmentation. ^[2]		
4.2	Preprocessing pour U-Net	9	Parmi les architectures de réseaux de neurones convolutifs (CNN) proposées pour la segmentation sémantique, l'U-Net, présenté en 2015 est l'une des plus populaires et des plus performantes.		
4.2.1	Augmentation des données	9	Ce modèle a donné par la suite naissance à de		
4.3	Expériences avec U-Net	9			
4.4	Variantes de U-Net	10			
4.5	Pipeline de spécialisation par clustering	10			
4.5.1	Clustering K-means	10			
4.5.2	Classification CNN pour prédiction de cluster	10			
4.5.3	Modèles U-Net spécialisés par cluster	11			
4.5.4	Aggrégation des résultats	13			
5	Résultats	14			
5.1	Preprocessing pour U-Net	14			
5.1.1	Augmentation des données	14			

nombreuses variantes, et en particulier à un framework particulièrement puissant: nnU-Net^[1]. Le nnU-Net est un framework de segmentation d'images biomédicales basé sur l'U-Net, qui a été présenté en 2019. Le nnU-Net est considéré comme une grande avancée dans le domaine de la segmentation d'images biomédicales en raison de sa capacité à fournir des résultats de segmentation précis et robustes sans nécessiter de modifications importantes de l'architecture ou des hyperparamètres.

1.2 Objectifs

Si nnU-Net est en effet très puissant, nous nous passerons de ce framework dans ce pro-

jet. L'objectif sera de mettre en place par nous-même un U-Net, et éventuellement une variante et une pipeline complète pour réaliser de la segmentation médicale.

Dans ce projet, nous verrons:

1. Le fonctionnement de base d'un U-Net sur des exemples très simples
2. L'influence de différents paramètres sur ses performances
3. Quelles méthodes peuvent nous permettre d'améliorer la performance du modèle

2 Présentation du dataset

Le dataset ([Kaggle Brain Tumor Segmentation](#)), est un ensemble de données destiné à la segmentation de tumeurs cérébrales en 2D. Il contient 3064 paires d'images IRM cérébrales et leurs masques binaires correspondants indiquant la présence de tumeurs.

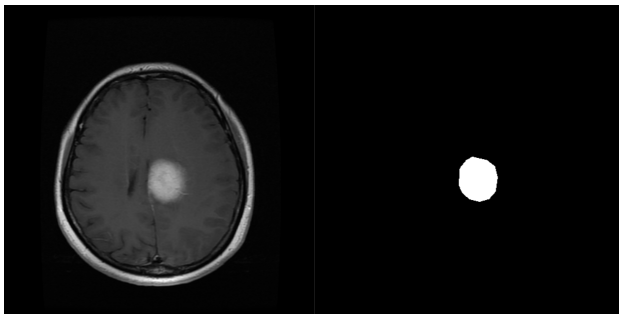


Figure 1: Exemple d'image IRM cérébrale et de son masque binaire correspondant

Certaines difficultés, nous le verrons, sont à prendre en compte, notamment:

Des différences d'intensité entre les tumeurs, avec certaines qui s'affichent très claire et sont donc très facilement visible et segmentable:

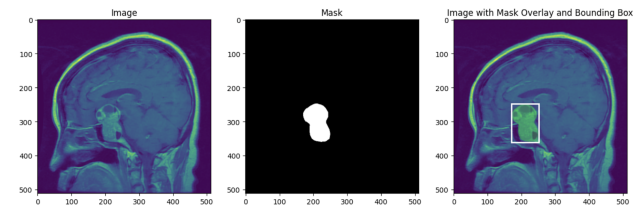


Figure 2: Exemple d'image IRM cérébrale et de son masque binaire correspondant

Et d'autres qui le sont beaucoup moins:

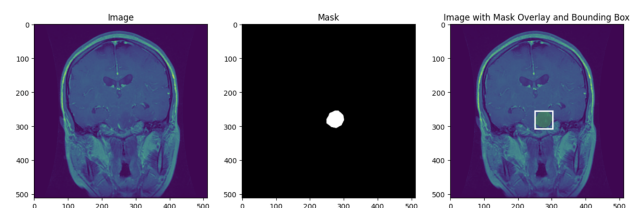


Figure 3: Exemple d'image IRM cérébrale et de son masque binaire correspondant

On peut résumer les caractéristiques chiffrées du dataset à l'aide des graphiques suivants:

On remarque un certain déséquilibre des tailles de tumeurs, avec une majorité de tumeurs petites relativement à la plage de tailles de tumeurs disponibles dans le dataset, comme en témoigne la figure suivante:

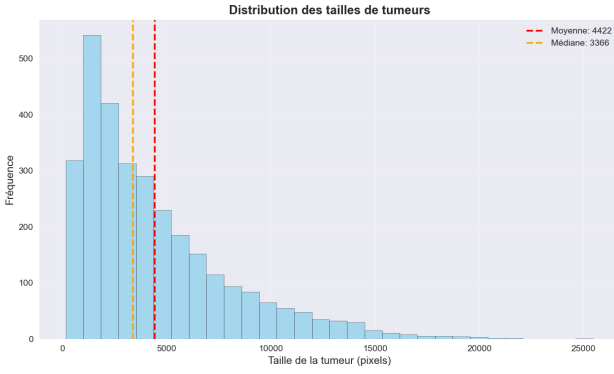


Figure 4: Exemple d'image IRM cérébrale et de son masque binaire correspondant

On remarque cependant que le dataset n'est composé que d'images contenant des tumeurs:

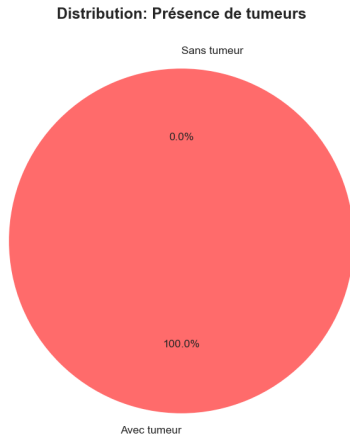


Figure 5: Distribution de la présence ou non d'une tumeur

Il faudra prendre cela en considération pour nos conclusions sur les performances du modèle: il s'agira d'un modèle entraîné à segmenter des tumeurs **sachant qu'il y a présence d'au moins une tumeur sur l'image**.

On peut aussi s'intéresser à l'excentricité des tumeurs, i.e. leur "allongement" si on les modélise comme étant des ellipses:

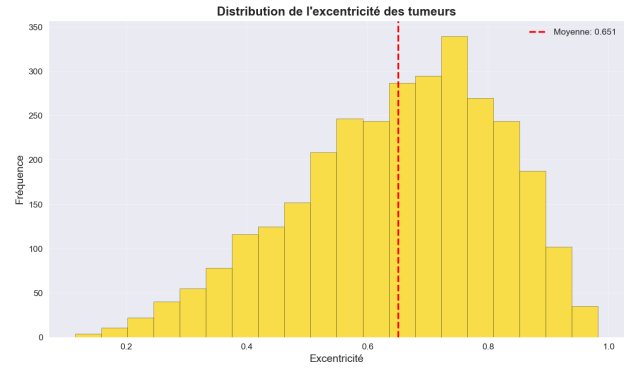


Figure 6: Distribution de l'excentricité des tumeurs du dataset

On remarque que le dataset donne une plage étendue d'excentricité, ce qui pourra permettre de généraliser correctement sur de nombreuses formes de tumeurs.

La solidité, qui mesure la convexité des formes géométriques, est pour la grande majorité proche de 1, c'est-à-dire proche d'une convexité parfaite. On remarque cependant l'existence de certaines tumeurs à géométrie non convexe:

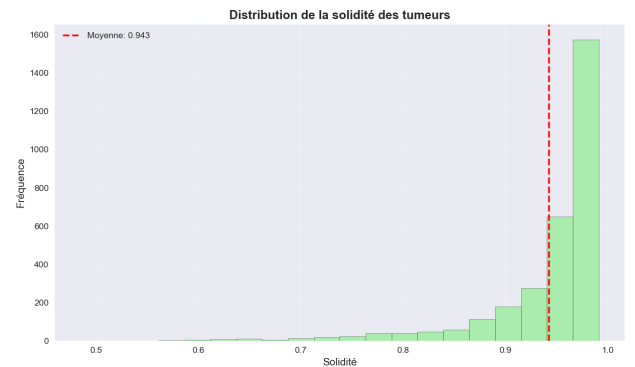


Figure 7: Distribution de la solidité des tumeurs du dataset

Il faudra prendre en compte cela dans les phases de la création du modèle plus avancées, avec des tumeurs de solidité "inhabituelles" qui devront être peut-être traitées différemment.

On peut résumer ces données dans un tableau de valeurs en fin de partie.

Table 1: Statistiques descriptives du dataset

Statistique	tumor_area	tumor_ratio	mean_intensity	perimeter	eccentricity	solidity	compactness	num_components
Nombre	3064	3064	3064	3064	3064	3064	3064	3064
Moyenne	4422	0.017	88.33	253	0.651	0.943	0.778	1.0
Écart-type	3619	0.014	29.15	118	0.167	0.062	0.128	0.0
Minimum	163	0.001	16.38	47	0.115	0.485	0.238	1.0
Q1 (25%)	1706	0.007	63.99	163	0.541	0.935	0.725	1.0
Médiane (50%)	3366	0.013	89.65	232	0.670	0.967	0.819	1.0
Q3 (75%)	6058	0.023	110.16	319	0.778	0.977	0.868	1.0
Maximum	25461	0.097	186.40	1147	0.983	0.992	1.003	2.0

3 Présentation de U-Net

Modèle de base de segmentation

3.1 Architecture U-Net

L'architecture U-Net^[3] tire son nom de sa forme caractéristique en "U" composée de deux parties symétriques :

1. Chemin contractant (encodeur) - partie gauche du U
2. Chemin expansif (décodeur) - partie droite du U

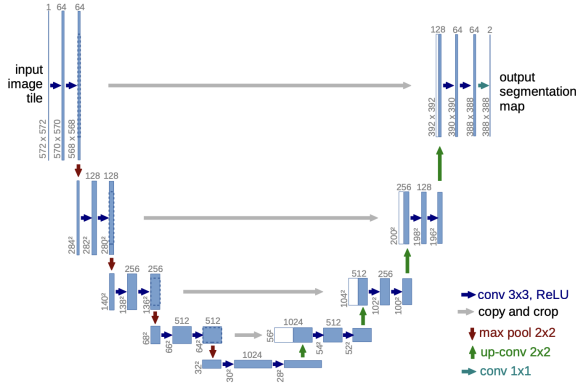


Figure 8: Schéma de l'architecture d'un U-Net

3.1.1 Chemin contractant (encoder)

Le chemin contractant suit l'architecture classique d'un CNN. Il est composé d'une succession d'étapes d'échantillonnage formée de:

2 convolutions $3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{max pooling } 2 \times 2$

Le nombre de canaux est doublé à chaque étape de sous-échantillonnage (voir Figure 2).

Cette étape a pour objectif de capturer le con-

texte de l'image. Plus précisément, l'idée est d'extraire du contexte à différentes échelles et de façon progressive. Le contexte fait ici référence à l'information spatiale globale qui permet d'obtenir de la compréhension sur l'image.

Plus clairement, le réseau, durant cette phase, capture des caractéristiques en formant de nouvelles représentations de l'image en extrayant des caractéristiques grâce à des "filtres" dont les paramètres sont ce qui est appris par le modèle.

3.1.2 Chemin expansif (decoder)

Le chemin expansif constitue la partie droite de l'architecture en U et suit une approche inverse au chemin contractant. Il remplace le max pooling par l'upsampling (sur-échantillonnage) pour reconstruire progressivement la résolution spatiale originale.

Chaque étape du chemin expansif est composée de:

upsampling $\rightarrow$ convolution $2 \times 2 \rightarrow$ concaténation avec la carte de caractéristiques correspondante du chemin contractant $\rightarrow$ 2 convolutions $3 \times 3 \rightarrow \text{ReLU}$

Le nombre de canaux est divisé par deux à chaque étape d'upsampling, restaurant progressivement la résolution spatiale tout en affinant les représentations (voir Figure 2).

Cette étape a pour objectif de permettre

une localisation précise des structures à segmenter. Plus précisément, l'idée est de combiner le contexte global capturé lors de la phase contractante avec les détails fins de l'image pour produire une segmentation pixel-parfaite. La localisation fait ici référence à la capacité du réseau à déterminer avec précision les contours et la position exacte de chaque structure dans l'image originale.

L'innovation clé réside dans les connexions de saut (skip connections) qui concaténent les cartes de caractéristiques du chemin contractant avec celles du chemin expansif. Ces connexions permettent de récupérer l'information spatiale fine qui aurait été perdue lors des opérations de sous-échantillonnage, créant ainsi un équilibre optimal entre compréhension contextuelle globale et précision de localisation locale.

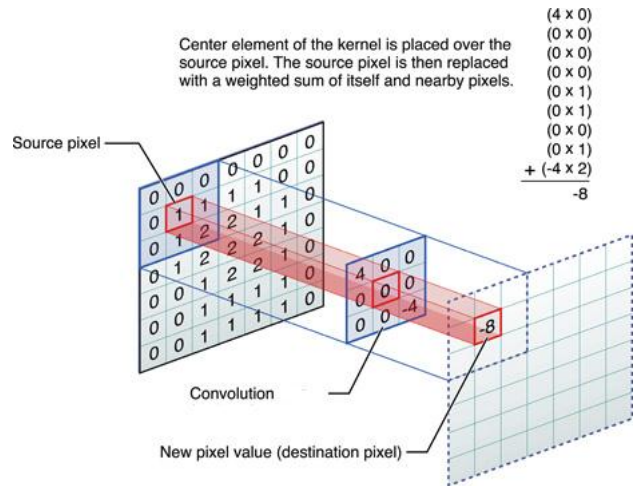


Figure 9: Schéma du principe d'une convolution*

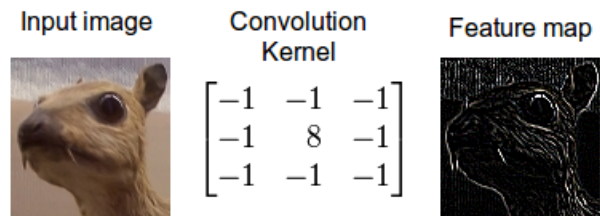


Figure 10: Exemple du principe d'une convolution†

3.1.3 Convolutions

Les convolutions constituent l'opération fondamentale des réseaux de neurones convolutifs. Une convolution applique un filtre (ou noyau) de petite taille sur l'ensemble de l'image d'entrée. Ce filtre "glisse" sur l'image pixel par pixel, effectuant à chaque position un produit scalaire entre ses valeurs et les pixels de l'image. L'objectif est d'extraire des caractéristiques spécifiques de l'image : détection de contours, de textures, de formes, etc. Les valeurs du filtre sont les paramètres appris par le réseau durant l'entraînement. Chaque filtre est spécialisé dans la détection d'un type particulier de motif.

3.1.4 Max Pooling

Le max pooling est une opération de sous-échantillonnage qui réduit la taille spatiale des cartes de caractéristiques. Il divise l'image en régions non-chevauchantes (typiquement 2×2) et ne conserve que la valeur maximale de chaque région. Cette opération permet de :

- Réduire la dimension des données (efficacité computationnelle)
- Augmenter le champ récepteur des couches suivantes
- Introduire une invariance aux petites translations
- Capturer l'information la plus saillante de chaque région

*<https://medium.com/@bdhuma/6-basic-things-to-know-about-convolution-daef5e1bc411>

†<https://developer.nvidia.com/discover/convolution>

3.1.5 Upsampling

L'upsampling est l'opération inverse du pooling : elle augmente la taille spatiale des cartes de caractéristiques. Dans U-Net, cette opération permet de restaurer progressivement la résolution originale de l'image. L'upsampling peut être réalisé par différentes techniques comme l'interpolation ou la convolution transposée.

Cette étape est cruciale pour obtenir une segmentation à la résolution de l'image d'entrée, permettant une localisation précise des contours des objets à segmenter.

3.1.6 Upsampling

L'upsampling est l'opération inverse du pooling : elle augmente la taille spatiale des cartes de caractéristiques. Dans U-Net, cette opération permet de restaurer progressivement la résolution originale de l'image.

L'upsampling peut être réalisé par différentes techniques comme l'interpolation ou la convolution transposée.

Cette étape est cruciale pour obtenir une segmentation à la résolution de l'image d'entrée, permettant une localisation précise des contours des objets à segmenter.

Note : Nous avons regardé plusieurs vidéos YouTube pour mieux comprendre ces concepts, notamment "But what is a convolution?" de 3Blue1Brown et d'autres vidéos de la série sur le Machine Learning du même créateur, qui donnent des explications et des visuels abordables.

3.2 Variantes

Des nombreuses variantes innovantes du U-Net classique ont depuis été créées pour rendre cette architecture plus performante sur certains problèmes.

Nous avons implémenté deux de ces variantes

†<https://paperswithcode.com/method/unet>

dans notre projet: Attention U-Net et U-Net++ (aussi nommé Nested U-Net).

3.2.1 U-Net++ (Nested U-Net)

U-Net++ est une évolution de U-Net qui introduit des connexions de saut denses et imbriquées pour améliorer la propagation des gradients et la fusion des caractéristiques multi-échelles.

Au lieu de connexions directes entre encoder et decoder, U-Net++ introduit des sous-réseaux imbriqués qui connectent progressivement les caractéristiques de différentes résolutions. Ces sous-réseaux créent des chemins de longueurs variables entre les couches d’encodage et de décodage.

Les nœuds de connexion intermédiaires effectuent des convolutions supplémentaires avant la concaténation, permettant une meilleure intégration des informations multi-échelles. Chaque niveau de résolution bénéficie ainsi d'informations pré-traitées provenant de plusieurs échelles.

Cette architecture améliore la segmentation d’objets de tailles variables et réduit l’écart sémantique entre les caractéristiques de l’encoder et du decoder, conduisant généralement à de meilleures performances au prix d’une complexité computationnelle accrue.

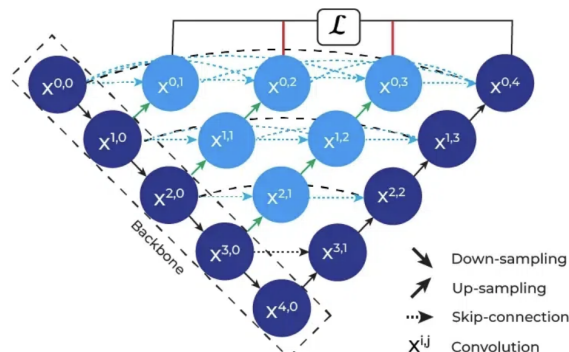


Figure 11: Schéma de l'architecture de U-Net++[‡]

3.2.2 Attention U-Net

Attention U-Net intègre des mécanismes d'attention dans l'architecture U-Net classique pour améliorer la focalisation du modèle sur les régions pertinentes de l'image.

Des modules d'attention sont ajoutés aux connexions de saut entre l'encodeur et le decoder. Les caractéristiques de l'encodeur sont "filtrées", pour supprimer les informations non pertinentes et amplifier les régions d'intérêt.

Cette architecture permet au modèle de se concentrer automatiquement sur les structures anatomiques d'intérêt tout en supprimant le bruit de fond, améliorant particulièrement efficace pour la segmentation

d'objets de petite taille ou dans des images avec un arrière-plan complexe, sans augmentation significative du nombre de paramètres.

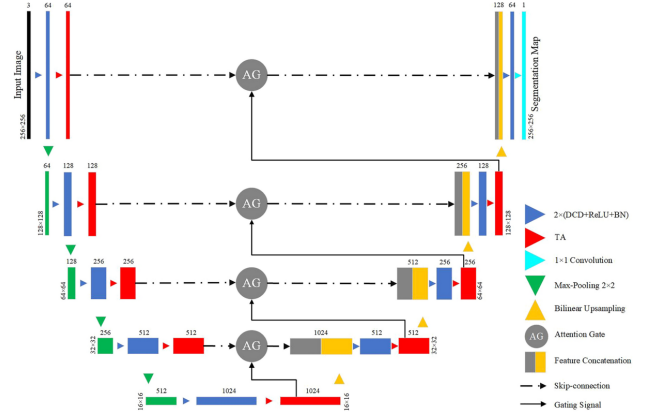


Figure 12: Schéma de l'architecture de Attention U-net^s

4 Méthodologie

Description la pipeline globale et de l'approche avec laquelle nous l'avons produite, sans résultats chiffrés détaillés

4.1 Vue d'ensemble

L'ensemble de la pipeline finale mise en place peut être résumé dans la figure suivante.

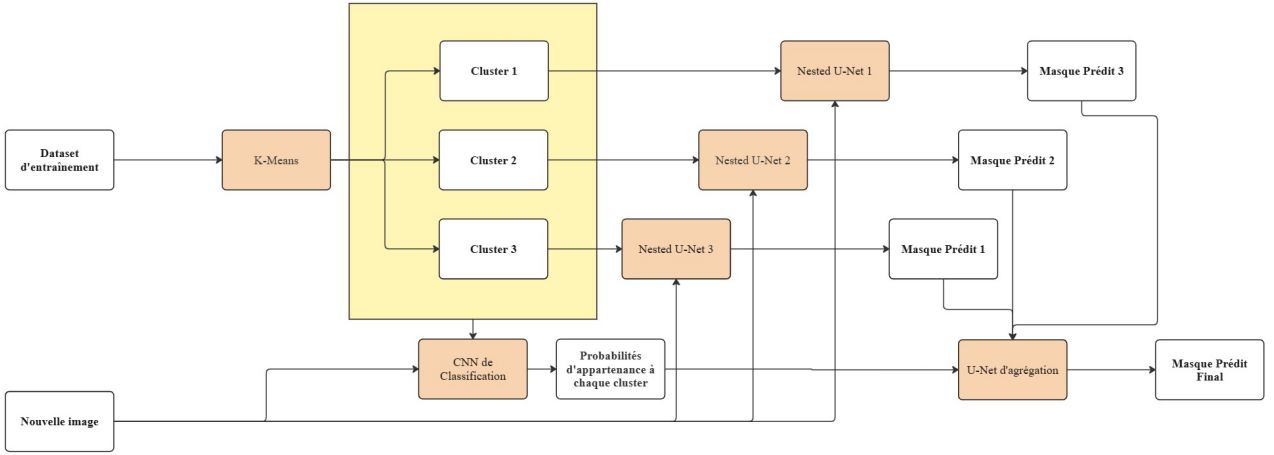


Figure 13: Diagramme de la pipeline complète

Après avoir entraîné différents modèles, en variant les hyperparamètres et la configuration du preprocessing, nous avons conclu que les Attention U-net étaient la meilleure architecture pour la tâche de segmentation sur les images du dataset.

Nous avons ensuite cherché à mettre en place une pipeline plus sophistiquée pour améliorer les performances du modèle. Nous avons entraîné un K-Means sur l'ensemble des données d'entraînement augmenté. Les données d'entrées du K-Means sont des caractéristiques de la partie des images en dessous de leur masque associé, ce qui nous permet d'avoir des clusters représentatifs de tumeurs aux caractéristiques similaires.

Les clusters ont servi à entraîner trois Attention U-Net spécialisés pour certains types de tumeurs. Ces Attention U-net ont permis d'avoir des performances bien supérieures sur leur cluster respectifs que l'Attention U-Net non spécialisé, entraîné initialement sur l'ensemble des données. Cependant, les nouvelles images n'ont évidemment pas de masque associé, et nous ne pouvons donc pas faire d'association entre celles-ci et un cluster pour faire une prédiction directement à l'aide d'un des modèles spécialisés.

Nous avons donc entraîné un réseau de neurones convolutionnel de classification, dont l'objectif est d'associer une nouvelle image à des probabilités d'appartenance à chaque clusters. Nous avons jugé les performances suffisamment bonnes pour intégrer ce modèle à la pipeline finale.

Cela nous permet, une fois avoir réalisé des predictions à l'aide des trois modèles spécialisés, d'agréger intelligemment les résultats pour obtenir le masque prédit final. L'aggrégation est réalisée à l'aide d'un U-Net classique, prenant en entrée 4 canaux: l'image d'origine en niveaux de gris, et un canal par masque prédit d'intensité pondérée par la probabilité d'appartenance à chaque cluster.

Comme nous le verrons plus tard, pensons pouvoir trouver une meilleure pipeline, notamment au niveau de la méthode d'aggrégation qui ne semble pas satisfaisante aussi bien au niveau des métriques d'évaluation, que de l'observation comparative des trois masques prédits par les modèles spécialisés et du masque finale prédit associé.

Cependant, au prix d'un coût computationnel plus élevé, la pipeline donne des résultats déjà très satisfaisants, et supérieurs à l'utilisation d'un U-Net seul.

4.2 Preprocessing pour U-Net

4.2.1 Augmentation des données

L'augmentation des données est une technique fondamentale en apprentissage automatique qui consiste à générer artificiellement de nouvelles données d'entraînement à partir du dataset existant. Elle permet d'augmenter la taille et la diversité du jeu de données sans collecte supplémentaire.

L'augmentation des données dans le cas des images est plutôt simple car les transformations sont pour beaucoup observables et compréhensibles directement.

L'augmentation des données permet de réduire le surapprentissage en exposant le modèle à plus de variabilité, d'améliorer donc la généralisation sur des données non vues, et dans certains cas peut équilibrer les classes en cas de déséquilibre dans le dataset

Dans notre projet, nous avons créé une fonction spécifique pour créer de nouvelles images et les enregistrer dans un dossier spécifique. Cette fonction permet de faire, au choix et de façon entièrement paramétrable des rotations, des translations, des changements d'échelle, des retournements verticaux et horizontaux, et des déformations plastiques. La fonction nous permet de passer d'un dataset de 3064 paires image/masque à un dataset de $3064 * n$ paires.

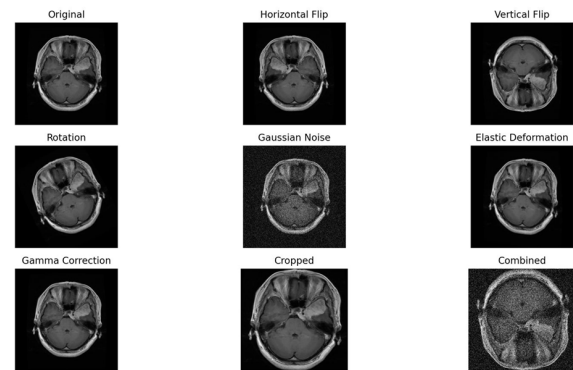


Figure 14: Exemples de transformations sur une image du dataset

L'augmentation des données est particulièrement utile dans notre cas, car comme vu dans l'introduction de la section méthodologie, et comme nous le verrons après, nous avons mis en place des modèles spécialisés sur des sous-ensembles du dataset, et nous avons besoin d'avoir toujours un volume suffisamment grand de données d'entraînement.

4.3 Expériences avec U-Net

Dans un premier temps, nous avons implémenté la version vanilla de l'architecture U-Net. Nous avons réalisés plusieurs tests sur un même dataset augmenté de façon arbitraire au début.

Les tests nous ont permis de conclure sur la meilleure taille pour le modèle: on remarque d'une part qu'un modèle a tendance à gagner en performances lorsqu'on rend le réseau

plus profond, et qu'on perd des performances en réduisant le nombre de neurones de la première couche (y compris en augmentant le nombre de couches total).

Cependant, on remarque qu'on ne peut pas ajouter une couche de 2048 neurones, car le temps d'entraînement devient trop grand et que le but de ce projet étant de tester différentes approches, nous ne pouvons pas nous permettre d'entraîner les modèles pendant des heures.

L'architecture retenue a alors été un ensemble de couche que l'on peut décrire comme [64, 128, 256, 512, 1024], qui semble offrir le meilleur compromis entre nombre de paramètres et performances. *(Le détail des résultats sera donné dans la prochaine partie).*

On a alors cherché à savoir à quel point on pouvait réduire la taille des images avant de perdre trop de performances. La perte de qualité des prédictions reste maîtrisée pour une baisse de la résolution de 512*512 à 128*128 pour l'ensemble du dataset. Tous les tests par la suite ont alors été faits avec cette résolution.

Enfin, nous avons réalisé des tests en utilisant différentes fonctions de pertes et en les combinant. Le résultat a été que la meilleure performance (mesurée par un Dice Score) est obtenue pour une perte combinant DiceLoss et Focal Loss, pondérés respectivement à 75% et 25%.

4.4 Variantes de U-Net

Nous avons alors gardé les conclusions obtenues pour la version vanilla du U-Net et implémenté un Attention U-Net et un Nested U-Net qui nous ont permis d'augmenter le Dice Score respectivement de 7,5% et 9,5% sur le split d'évaluation.

4.5 Pipeline de spécialisation par clustering

4.5.1 Clustering K-means

Nous avons utilisé K-Means pour former des clusters dans le but de créer des modèles spécialisés sur des types de tumeurs spécifiques. L'objectif est de créer des sous-groupes cohérents dans le dataset d'origine.

Des informations clés sur les tumeurs du dataset sont extraites par une fonction de pré-processing qui "découpe" les images autour du masque associé pour n'en analyser uniquement la tumeur. De là, nous extrayons des données liées à la forme et à l'intensité des pixels de la tumeur, que nous avons placées dans un tableau: surface de la tumeur, périmètre, excentricité, intensité minimum, maximum, moyenne, médiane, ...

Nous avons pour cahier des charges d'obtenir un nombre de cluster au plus de 5 pour limiter le nombre de modèles spécialisés à entraîner ensuite.

Au final, l'analyse de la courbe par la méthode du coude de l'inertie intra-cluster en fonction du nombre de clusters k a démontré que le nombre de clusters optimal était de 3.

Pour les tests suivants, nous avons créé un fichier .csv contenant l'identifiant de chaque doublet image/masque et leur cluster associé.

4.5.2 Classification CNN pour prédiction de cluster

Comme présenté dans l'introduction, les nouvelles images ne peuvent pas être associées à un cluster directement, car nous n'avons pas les images pour opérer la "découpe" dont il a été question dans la présentation de la démarche pour la mise en place de l'algorithme K-Means. Nous avons donc créé un modèle de prédiction de l'appartenance à un cluster.

Un réseau de neurones à convolution d'architecture relativement simple nous a per-

mis d'obtenir des résultats très satisfaisants: il adopte une architecture séquentielle composée de deux blocs convolutionnels suivis de couches entièrement connectées. La configuration par défaut utilise les paramètres suivants :

- **Entrée** : Images de taille 128x128 pixels (conformément à l'entrée pour k-means et pour les modèles de segmentation spécialisés)
- **Configuration des filtres** : [64, 128, 256]
- **Nombre de classes** : 3 (i.e. nombre de clusters)

4.5.3 Modèles U-Net spécialisés par cluster

Nous savions alors que nous serions à priori capable d'orienter les images vers un cluster, ou au moins d'avoir une probabilité d'appartenance à chaque cluster qui soit suffisamment fiable pour agréger les résultats si nécessaire. Nous avons donc pu nous lancer dans l'entraînement de trois U-Net++ (modèle ayant donné le meilleur résultat sur l'entraînement précédent) spécialisés sur les clusters en utilisant le .csv généré plus tôt.

Ces modèles spécialisés ont largement dépassé les performances du modèle de base, avec une augmentation du Dice Score allant de 2,5% à 9,5% environ sur les sous-ensembles d'évaluation.

Après observation des résultats, nous avons remarqué plusieurs choses intéressantes et assez inattendues:

1. Le modèle spécialisé sur le cluster sélectionné par le CNN de classification est loin d'être nécessairement celui qui donne la meilleure segmentation pour une image donnée;
2. Les images n'ayant pas au moins une segmentation de très bonne qualité parmi les trois proposées sont très rares.

Tout cela est illustré dans la figure de la page suivante qui permet d'avoir une bonne vue d'ensemble du potentiel d'une bonne aggrégation des résultats de ces modèles spécialisés.

Le challenge restant est alors la mise en place d'une méthode d'aggrégation des prédictions faites par ces modèles en un seul modèle qui soit performante.

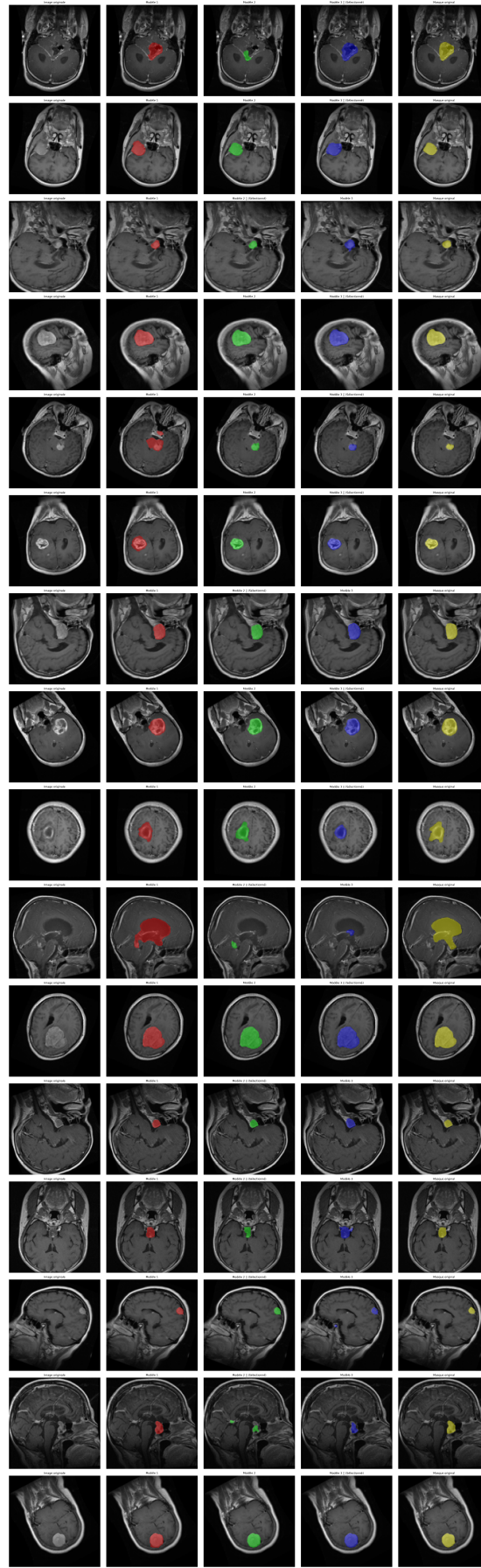


Figure 15: Dans la colonne gauche, images en entrée; Dans les 3 colonnes suivantes, les trois prédictions de masques faites par les modèles spécialisés; Dans la dernière colonne, le masque à prédire

4.5.4 Aggrégation des résultats

Pour l'aggrégation des résultats, nous avons tenté deux approches:

1. Aggrégation par votes et pondérations Nous avons tout d'abord utilisé une technique d'aggrégation basée sur le vote majoritaire de plusieurs modèles ou prédictions. Pour chaque pixel de l'image, chaque modèle (ou augmentation d'inférence) prédit la présence ou non de tumeur. La segmentation résultante est obtenue selon les choix suivants:

Vote majoritaire strict: pour chaque pixel, la classe la plus souvent prédite est retenue.

Vote majoritaire plus souple: Au moins deux modèles, trois modèles, moyenne et seuils personnalisés par score de confiance ou probabilité prédite par le modèle de classification

Nous avons expérimenté avec différents seuils pour maximiser la métrique de segmentation globale (Dice, IoU) sur un jeu de validation.

Cette approche vise à exploiter la diversité des modèles (ou des augmentations) pour améliorer la robustesse des résultats, en réduisant l'influence des erreurs isolées.

Comme nous l'avions attendu, cette méthode c'est pas adaptée à notre problème et donne des résultats de moins bonne qualité

2. Aggrégation par Unet Nous avons finalement testé une stratégie d'aggrégation basée sur un U-Net supplémentaire.

Cette approche consiste à : Concaténer les différentes prédictions (ou probabilités) issues des modèles ou augmentations en canaux d'entrée supplémentaires, puis à entraîner un U-Net prenant comme entrée cette pile d'informations multi-source.

L'idée est que le réseau puisse "apprendre" les combinaisons les plus pertinentes entre les variantes, pour produire un masque final potentiellement meilleur que n'importe quelle prédiction prise individuellement.

Ce réseau d'aggrégation agit comme un méta-modèle de post-traitement, cherchant à exploiter de façon non-linéaire les complémentarités entre les différentes sorties. Nous avons comparé cette approche à la méthode de votes pondérés décrite ci-dessus afin de déterminer dans quels cas elle apportait une amélioration significative.

5 Résultats

Visualisations comparatives des sorties, métriques, ...

5.1 Preprocessing pour U-Net

5.1.1 Augmentation des données

Nous n'avons pas fait d'études comparatives des performances sur l'augmentation, car nous nous sommes rendu compte assez vite que, puisque nous divisons les données en clusters, nous avons intérêt à varier les augmentations de sorte à augmenter suffisamment le dataset de façon variée pour avoir suffisamment de données pour entraîner les modèles spécialisés.

5.2 Expériences avec U-Net

La première série d'expérimentations visait à évaluer l'influence de la profondeur du réseau sur les performances de segmentation. Nous avons testé différentes architectures en faisant varier le nombre de couches et la résolution des images d'entrée.

Table 2: Impact de la résolution (Dice Score)

Architecture	256×256	128×128
[64,128,256,512]	0.65	0.63
[64,128,256,512,1024]	0.75	0.71

Les résultats montrent clairement que l'ajout d'une couche supplémentaire (1024 filtres) améliore significativement les performances, avec un gain de 0.10 point de Dice Score pour la résolution 256×256 et de 0.08 point pour la résolution 128×128.

Nous observons également que la réduction de résolution de 256×256 à 128×128 n'entraîne qu'une perte modérée de performance (0.04 point), ce qui nous a conduits à privilégier la résolution 128×128 pour les expérimentations suivantes afin d'optimiser les temps de calcul.

5.2.1 Exploration d'architectures alternatives

Nous avons ensuite testé des architectures débutant par des couches de taille plus réduite pour évaluer l'impact d'une approche plus progressive :

Table 3: Architectures alternatives (128×128)

Architecture	Dice
[32,64,128,256,512]	0.55
[32,64,128,256,512,1024]	0.01

Ces expérimentations révèlent que commencer avec des couches de plus petite taille (32 filtres) dégrade significativement les performances. L'architecture la plus profonde avec cette approche a même échoué à converger, suggérant des problèmes de gradient ou de capacité du modèle.

5.2.2 Optimisation de la fonction de perte

La sélection de la fonction de perte appropriée est cruciale pour la segmentation d'images. Nous avons évalué plusieurs fonctions sur l'architecture optimale identifiée précédemment : [64, 128, 256, 512, 1024] avec une résolution 128×128, sur 8 époques.

Table 4: Comparaison des fonctions de perte

Fonction de perte	Dice
BCEWithLogits	0.63
Dice Loss	0.72
Cross Entropy	0.025
IoU Loss	0.71
Focal Loss	0.70
DL×FL ($\alpha=0.5$)	0.70
DL×FL ($\alpha=0.25$)	0.72
DL×FL ($\alpha=0.75$)	0.72

5.2.3 Configuration finale retenue

Sur la base de ces expérimentations, nous avons retenu la configuration suivante pour notre modèle U-Net :

- **Architecture** : [64, 128, 256, 512, 1024] filtres
- **Résolution** : 128×128 pixels
- **Fonction de perte** : Dice Loss ou IoU Loss (0.72)

Cette configuration représente le meilleur compromis entre performance (Dice Score de 0.72) et efficacité computationnelle. L'échec catastrophique de la Cross Entropy (score de 0.025) confirme l'importance du choix de la fonction de perte pour les tâches de segmentation, où les métriques géométriques comme Dice et IoU sont plus adaptées que les métriques de classification traditionnelles.

Pour illustrer, voici ce à quoi peuvent ressembler des prédictions aillant un Dice Score d'environ 0.72:

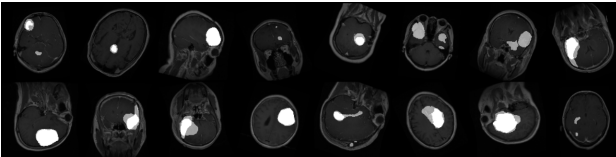


Figure 16: Exemple de prédictions de Dice Score 0.72 (1)

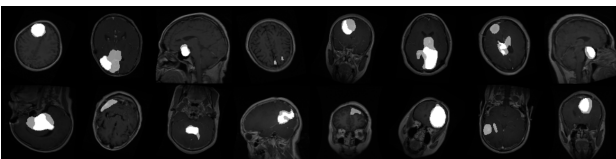


Figure 17: Exemple de prédictions de Dice Score 0.72 (1)

Pour comprendre ces images, il faut simplement comprendre que l'on superpose ici le masque prédit et le masque réel, les deux avec une transparence de 0.5. Ainsi, on voit les pixels correctement prédits sur les zones blanches, et les zones grises sont les zones de non-superposition de ces deux masques.

5.3 Variantes de U-Net

Après avoir établi une configuration de base optimale, nous avons exploré différentes variantes architecturales de U-Net pour améliorer davantage les performances de segmentation.

5.3.1 Attention U-Net

L'Attention U-Net intègre des mécanismes d'attention qui permettent au modèle de se concentrer sur les régions les plus pertinentes lors de la segmentation. Sans autre modification que l'architecture du modèle, nous avons testé cette variante avec notre configuration optimale.

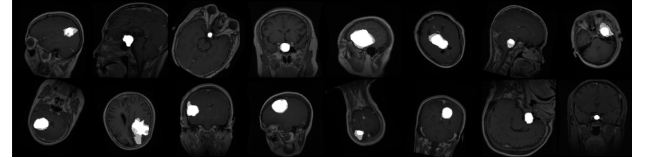


Figure 18: Exemple de prédictions de Dice Score 0.74 (1)

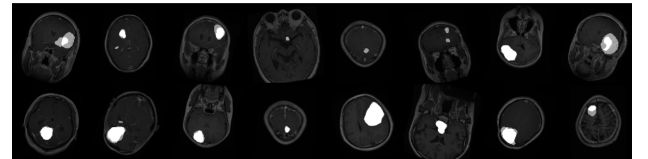


Figure 19: Exemple de prédictions de Dice Score 0.74 (2)

Table 5: Performance Attention U-Net

Modèle	Dice Score
Attention U-Net (DL×FL $\alpha=0.75$)	0.74

Cette amélioration de 0.02 point par rapport au U-Net standard (0.72) démontre l'efficacité des mécanismes d'attention pour la segmentation, bien que la différence ne soit pas transcendente.

5.3.2 Nested U-Net avec Squeeze-and-Excitation

Nous avons ensuite évalué le Nested U-Net (U-Net++) qui utilise des connexions denses et

des modules Squeeze-and-Excitation. Cette architecture plus sophistiquée a été testée sur 30 époques avec un scheduler de taux d'apprentissage (Step LR, facteur 0.1 toutes les 10 époques).

Table 6: Nested U-Net - Architectures (30 époques, 128×128)

Architecture	Dice Score
[32,64,128,256,512]	0.7980
[64,128,256,512,1024]	0.8108
[32,64,128,256,512,1024]	0.8157

Les résultats montrent une amélioration substantielle par rapport au U-Net standard, avec le meilleur score de 0.8157 pour l'architecture la plus profonde.

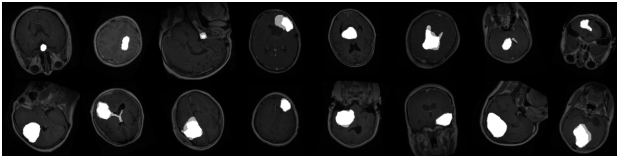


Figure 20: Exemple de prédictions de Dice Score 0.82 (1)

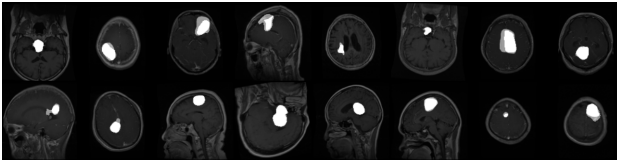


Figure 21: Exemple de prédictions de Dice Score 0.82 (2)

5.3.3 Entraînement complet

Pour valider ces résultats prometteurs, nous avons mené un entraînement complet sur 100 époques avec la meilleure configuration identifiée, en augmentant également la résolution à 128×128 pixels.

Table 7: Nested U-Net - Entraînement complet

Configuration	Dice Score
Nested U-Net 100 époques (256×256)	0.835

Ce résultat final de 0.835 représente une amélioration significative de 0.115 point par rapport à notre U-Net de base (0.72), démontrant l'efficacité de l'architecture Nested U-Net combinée à un entraînement plus long et une résolution supérieure.

L'évolution progressive des performances (U-Net standard: 0.72 $\rightarrow$ Attention U-Net: 0.74 $\rightarrow$ Nested U-Net: 0.83) confirme l'intérêt des architectures plus sophistiquées pour notre tâche de segmentation.

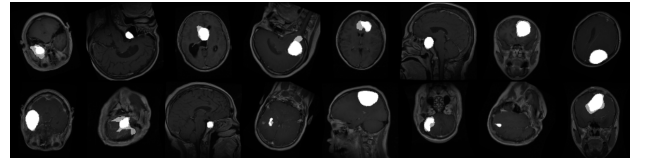


Figure 22: Exemple de prédictions de Dice Score 0.835 (1)

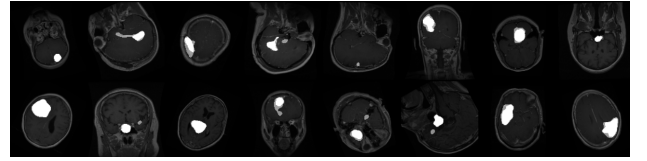


Figure 23: Exemple de prédictions de Dice Score 0.835 (2)

5.4 Pipeline de spécialisation par clustering

5.4.1 Clustering K-means

Pour déterminer le nombre optimal de clusters dans notre approche de spécialisation, nous avons appliqué la méthode du coude sur notre dataset d'images de tumeurs cérébrales. Cette technique consiste à analyser l'évolution de l'inertie intra-cluster (somme des carrés des distances) en fonction du nombre de clusters k .

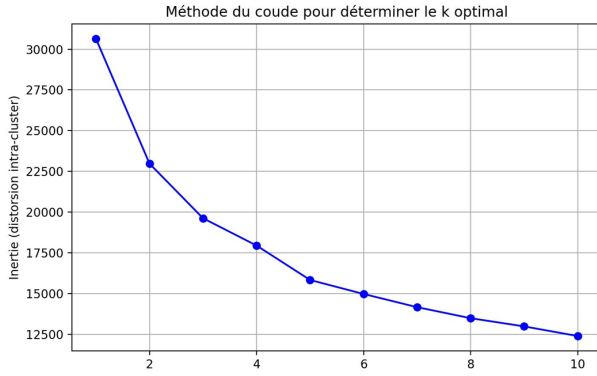


Figure 24: Méthode du coude pour la détermination du nombre optimal de clusters

On observe de plus une bonne séparation de l'espace sur différents exemples d'ensembles de 3 caractéristiques (on ne peut en effet pas afficher un espace de plus de trois dimensions ce qui nous oblige à ne sélectionner que trois des caractéristiques du dataset d'entrée de k-means pour illustrer.

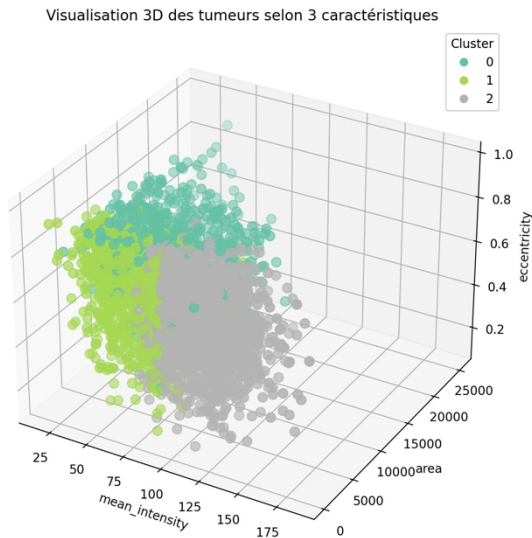


Figure 25: Méthode du coude pour la détermination du nombre optimal de clusters

5.4.2 Classification CNN pour prédiction de cluster

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pel-

lentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5.4.3 Modèles U-Net spécialisés par cluster

Après avoir identifié trois clusters distincts par la méthode K-means, nous avons procédé à l'entraînement de modèles Nested U-Net spécialisés pour chaque cluster. Chaque modèle utilise la configuration optimale précédemment établie dans notre étude comparative :

- **Architecture** : Nested U-Net avec modules Squeeze-and-Excitation
- **Configuration des filtres** : [64, 128, 256, 512, 1024]
- **Résolution d'images** : 128×128 pixels
- **Taille de batch** : 32
- **Fonction de perte** : Dice Loss × Focal Loss ($\alpha=0.75$)
- **Optimiseur** : Adam avec scheduler Step LR (facteur 0.1 toutes les 10 époques)

Cette approche de spécialisation permet à chaque modèle de se concentrer exclusivement sur les caractéristiques morphologiques spécifiques à son cluster, optimisant ainsi l'apprentissage des motifs de segmentation particuliers à chaque type de tumeur.

Table 8: Performance des modèles Nested U-Net spécialisés par cluster

Cluster	Dice Score
0	0.85
1	0.92
2	0.90

Les résultats obtenus démontrent l’efficacité de notre approche de spécialisation. Tous les modèles spécialisés surpassent significativement la performance du modèle généraliste Nested U-Net (0.83), avec une amélioration moyenne de 0.06 point de Dice Score.

Le cluster 1, présente les meilleures performances (0.92), suggérant que ce type de tumeur bénéficie particulièrement de la spécialisation. Le cluster 0, malgré un score légèrement inférieur (0.85), montre néanmoins une amélioration substantielle par rapport au modèle généraliste, indiquant que même les tumeurs de petite taille profitent de cette ap-

proche ciblée.

La dernière figure de la partie méthodologie montre les résultats de cette stratégie avant aggrégation.

Cette stratégie de spécialisation valide notre hypothèse selon laquelle la segmentation de tumeurs cérébrales peut être améliorée en adaptant l’architecture aux caractéristiques morphologiques spécifiques de chaque type tumoral.

5.4.4 Aggrégation des résultats

Le modèle retenu a été le U-Net supplémentaire qui permet d’améliorer légèrement les résultats obtenus par un modèle unique.

Après avoir utilisé une approche Bayésienne avec optuna, nous avons remarqué que toutes les architectures testées avaient des résultats de 85% et 86,5% pour le Dice-Score final.

6 Conclusions

Les résultats obtenus sont globalement satisfaisants et montrent un caractère qualitatif encourageant pour la suite du projet. Nous sommes néanmoins conscients des marges d’amélioration possibles, tant sur le plan méthodologique que sur celui de la validation. Plusieurs pistes pourraient être explorées afin de renforcer la robustesse et la généricité du modèle proposé :

- L’application d’approches bayésiennes à l’ensemble des modèles permettrait une meilleure quantification de l’incertitude et une interprétabilité accrue des résultats.
- L’expérimentation avec d’autres ensembles de données pour la phase de *k-means* offrirait une meilleure évaluation de la robustesse des clusters.
- La recherche d’une méthode d’agrégation plus performante pourrait améliorer la synthèse des différentes sources d’information.
- La mise en place d’une procédure de *cross-validation* systématique permettrait d’obtenir une estimation plus fiable des performances du modèle.
- Il serait pertinent d’isoler une partie du jeu de données pour une validation finale indépendante, afin de limiter le risque de sur-apprentissage.
- Enfin, enrichir le dataset en intégrant des bases de données supplémentaires — notamment celles contenant des exemples sans tumeur — permettrait de renforcer la généralisation et la pertinence du modèle dans des conditions variées.

Des extensions futures pourraient ainsi systématiser ces axes afin d'améliorer la portée et la fiabilité des résultats présentés ici.

References

- [1] F. Isensee, J. Petersen, A. Klein, D. Zimmerer, P. F. Jaeger, S. Kohl, J. Wasserthal, G. Koehler, T. Norajitra, S. Wirkert, and K. H. Maier-Hein. nnu-net: Self-adapting framework for u-net-based medical image segmentation. *arXiv preprint arXiv:1809.10486*, 2018.
- [2] L. Maier-Hein, M. Eisenmann, A. Reinke, S. Onogur, M. Stankovic, P. Scholz, T. Arbel, H. Bogunovic, A. P. Bradley, A. Carass, C. Feldmann, A. F. Frangi, P. M. Full, B. van Ginneken, A. Hanbury, K. Honauer, M. Kozubek, B. A. Landman, K. März, O. Maier, K. Maier-Hein, B. H. Menze, H. Müller, P. F. Neher, W. Niessen, N. Rajpoot, G. C. Sharp, K. Sirinukunwattana, S. Speidel, C. Stock, D. Stoyanov, A. A. Taha, F. van der Sommen, C.-W. Wang, M.-A. Weber, G. Zheng, P. Jannin, and A. Kopp-Schneider. Why rankings of biomedical image analysis competitions should be interpreted with care. *Nature Communications*, 9(1):5217, 2018.
- [3] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *arXiv preprint arXiv:1505.04597*, 2015.